SGLang so với llama.cpp: chạy model đơn trên một card (RTX 3060)

Ngày đo 09/11/2026. Chỉ đo hiệu suất, không đo chất lượng câu trả lời. Mọi số liệu là số đo thực; phần suy luận nguyên nhân được gắn [ƯỚC TÍNH].

1. Kết luận

1. **Một luồng: hai engine gần như ngang nhau.** Qwen3-8B decode 62,9 tok/s (SGLang) so với 63,2 tok/s (llama.cpp); Qwen3-4B 107,7 so với 100,7 tok/s (SGLang nhanh hơn 7%). Prefill chênh dưới 10%.
2. **Nhiều luồng: SGLang hơn.** Với 4 luồng SGLang hơn 36% (8B: 241 so với 177 tok/s) đến 51% (4B: 405 so với 269 tok/s). Độ trễ token đầu khi 4 luồng chỉ 25–37 ms, so với 123–149 ms của llama.cpp. Với 2 luồng, hiệu số phụ thuộc vào slot mà llama.cpp chọn: SGLang hơn khoảng 9–12% khi llama.cpp dùng hai slot kề nhau (8B: 123 so với 113; 4B: 209 so với 186) và gấp 2,1–2,3 lần khi llama.cpp rơi vào hai slot không kề nhau, trường hợp xảy ra trong bài đo chính (xem báo cáo `BAO_CAO_LLAMACPP_THONG_LUONG_2_LUONG`).
3. **Điện năng mỗi token thấp hơn 6–13%** ở SGLang vì cùng công suất GPU nhưng sinh được nhiều token hơn mỗi giây.
4. **Đổi lại SGLang tốn nhiều tài nguyên và công sức khởi động hơn:** nạp mất 21–22 giây (llama.cpp 3–7 giây), chiếm gấp đôi VRAM vì cấp phát trước bộ nhớ KV (9,8 GB so với 4,9–7,0 GB), và cần trọng số HF/AWQ thay vì GGUF.
5. **Kết luận cho project:** với một card phục vụ nhiều người dùng cùng lúc, SGLang đáng cân nhắc hơn llama.cpp. Với một người dùng đơn lẻ không có khác biệt đáng kể. Bài đo này không bao gồm ghép GPU qua mạng (merged), là phần lõi của project.

2. Điều kiện đo

	llama.cpp (production)	SGLang
Phiên bản	llama-server build 11913 (CUDA)	SGLang 0.5.19
Định dạng model	GGUF Q4_K_M (4B 2,5 GB; 8B 5,0 GB)	AWQ 4-bit của Qwen (4B 2,7 GB; 8B 5,9 GB), nhân awq_marlin
Cấu hình	`-ngl 99 -fa on -c 16384 -np 4`, KV f16	context 16.384, `--max-running-requests 20`, `--mem-fraction-static 0.80`, fp16, attention triton, CUDA graph decode (tối đa 8 luồng)

- Phần cứng: Linux, một RTX 3060 12 GB, production tạm dừng trong lúc đo và đã khôi phục.
- Cùng một client đo cho cả hai (HTTP `/v1/chat/completions` dạng stream, tương thích OpenAI), cùng tokenizer Qwen3 nên số token prompt giống nhau (464 và 1.328 token). Mỗi phép đo lặp 3 lần, lấy trung vị; sinh 192 token.
- `--max-running-requests 20` lớn hơn số luồng đo để tránh lỗi tính hàng đợi của SGLang 0.5.19 (đã được sửa ở 0.5.23).

3. Kết quả

Qwen3-4B

Chỉ số	llama.cpp	SGLang	Thay đổi
Decode một luồng	100,7 tok/s	**107,7 tok/s**	+7%
Prefill 1.328 token	3.788 tok/s	**4.183 tok/s**	+10%
Độ giật p99 giữa hai token	10,5 ms	10,1 ms	−4%
Thông lượng 2 luồng song song	92,7 tok/s (slot không kề nhau); 186 tok/s (slot kề nhau)	**209,1 tok/s**	×2,26; +12%
Thông lượng 4 luồng song song	268,6 tok/s	**404,7 tok/s**	+51%
TTFT tệ nhất khi 4 luồng	149 ms	**25 ms**	−83%
Điện năng	1,62 J/token	**1,41 J/token**	−13%
Thời gian nạp model	**3,0 s**	22,0 s	×7
VRAM đỉnh	**4,9 GB**	9,8 GB	×2,0

Qwen3-8B

Chỉ số	llama.cpp	SGLang	Thay đổi
Decode một luồng	63,2 tok/s	62,9 tok/s	−0,5%
Prefill 1.328 token	2.534 tok/s	2.439 tok/s	−4%
Độ giật p99 giữa hai token	16,6 ms	16,8 ms	+1%
Thông lượng 2 luồng song song	60,0 tok/s (slot không kề nhau); 113 tok/s (slot kề nhau)	**123,0 tok/s**	×2,05; +9%
Thông lượng 4 luồng song song	177,4 tok/s	**240,9 tok/s**	+36%
TTFT tệ nhất khi 4 luồng	123 ms	**37 ms**	−70%
Điện năng	2,63 J/token	**2,48 J/token**	−6%
Thời gian nạp model	**7,0 s**	21,0 s	×3
VRAM đỉnh	**7,0 GB**	9,9 GB	×1,4

4. So sánh nguyên nhân

Hiện tượng	Nguyên nhân
Một luồng ngang nhau	Giải mã một token bị giới hạn bởi việc đọc trọng số từ bộ nhớ GPU; hai định dạng 4 bit có kích thước xấp xỉ nhau (2,5–2,7 GB và 5,0–5,9 GB) nên tốc độ giống nhau [ƯỚC TÍNH]
SGLang hơn khi 2–4 luồng	SGLang gom các luồng vào cùng lần tính (continuous batching, CUDA graph cho batch nhỏ) nên thông lượng tăng gần tuyến tính từ 1 lên 2 luồng. Ở llama.cpp, thông lượng 2 luồng trong bài đo chính (93 và 60 tok/s) thấp hơn một luồng đơn vì hai luồng rơi vào hai slot không kề nhau và bị giải mã lần lượt; khi ở slot kề nhau đạt 186 và 113 tok/s (nguyên nhân và số đo chi tiết trong báo cáo riêng)
TTFT khi 4 luồng của SGLang thấp hơn	Bộ lập lịch SGLang nhận các yêu cầu mới vào batch ngay, còn llama.cpp có 4 slot dùng chung một bộ nhớ KV nên các yêu cầu chen nhau [ƯỚC TÍNH]
Nạp lâu hơn	SGLang tải trọng số, cấp phát trước KV và ghi lại CUDA graph khi khởi động; llama.cpp chỉ ánh xạ file GGUF đã nằm trong bộ nhớ đệm của hệ điều hành
VRAM gấp đôi	`--mem-fraction-static 0.80` cấp trước 80% VRAM cho KV và trọng số, nên con số này là mức đặt trước, không phải mức dùng thực

5. Giới hạn

- Chỉ đo một card RTX 3060 trên Linux với hai model Qwen3 (4B, 8B), tối đa 4 luồng. Chưa đo 8–16 luồng, model MoE 30B, ghép GPU qua mạng hay Windows.
- Định dạng lượng tử khác nhau (AWQ so với GGUF Q4_K_M) nên chất lượng đầu ra có thể khác; chưa so sánh chất lượng.
- SGLang 0.5.19 chạy với nhân attention triton và sampling pytorch vì flashinfer không biên dịch được trong môi trường hiện tại; bản mới hơn hoặc flashinfer có thể nhanh hơn.

- Mỗi cấu hình chỉ chạy một lượt (3 lần lặp mỗi phép đo). Số 2 luồng của llama.cpp trong bài đo chính là trường hợp xấu (slot không kề nhau); số "slot kề nhau" lấy từ lần đo riêng sau đó.